E-COMMERCE BACKEND SYSTEM

Capstone Project



ENSP456 – New Age Programming Languages Lab

By

ANJALI (2301420021)
JIGYASA SINGH (2301420032)
BHUVEETA SAROHI (2301420011)
ARSHIYA TAHIM(2301420031)
DEV PAWAR(2301420028)
LAKSHAY PAWAR(2301420007)

Submitted to- Dr. Hemraj Kumawat(SOET)
Semester- 7th
Course - Bachelor of Technology in Computer Science and Engineering(Data Science)
Section - A
Submitted on - 22 October 2026

1. Problem Statement & Motivation

1.1 Problem Description

Modern e-commerce platforms must handle many tasks simultaneously: user management, product catalogs, inventory control, shopping carts, orders, payments, invoicing, and notifications. A simple monolithic backend can become a bottleneck: for example, generating an invoice or sending an email during checkout might tie up resources needed for processing the order. Similarly, updating a large codebase or scaling one part of the application requires redeploying the entire system. This can lead to slow response times and maintenance difficulties. The problem addressed here is designing an e-commerce backend where **time-critical operations are separated from background tasks**, improving concurrency and maintainability.

In this context, customers and administrators interact via a web or API client (e.g., Postman or a simple web page). Key operations include registering/logging in users, browsing products, managing a shopping cart, and completing checkout. After checkout, additional tasks such as generating an invoice, sending notifications, or updating recommendations can be handled independently.

1.2 Motivation

Splitting the workload leverages each language's strengths and creates a realistic microservice architecture. Go's concurrency model makes it ideal for the core transactional path, ensuring quick response for order checkout and inventory updates. Python, with its rich libraries and asyncio support, is well suited for IO-bound background tasks. Decoupling these via an event-driven approach ensures that user-facing performance is not degraded by slower operations.

1.3 Existing Solutions and Limitations

Large e-commerce companies often use complex microservices, queues, and cloud infrastructure, which are beyond the scope of an academic project. A simpler monolithic system (all code in one program) is easier to build but cannot easily demonstrate concurrency and independent scaling. This project occupies the middle ground: it does not attempt full production-grade features like real payment gateways or Kubernetes deployment, but it shows how splitting an application into

services (using Go and Python) can improve responsiveness and maintainability. By focusing on the planning stage (Phase I), we ensure a clear architecture before coding.

2. Proposed Solution & Language Justification

2.1 Project Overview

The system will consist of multiple backend services working together:

- A **Go-based API layer** that receives REST requests for users, products, inventory, carts, and orders.
- **Go microservices** for User, Product, Inventory, and Order management, each connected to a PostgreSQL database.
- **Redis Pub/Sub** as a lightweight message broker to deliver events such as OrderCreated.
- **Python background services** that subscribe to events: one generates invoice PDFs, one logs notifications, and one updates basic product recommendations.
- **Docker containers** to run each service in isolation for consistent deployment.

When a customer checks out, the Go Order Service performs the core transaction: it validates inventory, creates the order record, updates stock, and then publishes an OrderCreated event (containing order details) to a Redis channel. Python services asynchronously consume this event: the Invoice service generates a PDF invoice, the Notification service records a simulated notification, and the Recommendation service updates suggestion data. The customer receives an immediate order confirmation without waiting for the slower background tasks to finish.

2.2 Language Choice: Go and Python

We select **Go** and **Python** as the primary implementation languages. Go excels at concurrent network services. Python is chosen because it is widely used for background tasks and has rich libraries for tasks like PDF generation. No additional new language (e.g., F#, Clojure, Kotlin) is needed, keeping the implementation focused and moderate in complexity.

Justification for Go

- **Concurrency with Goroutines:** Go's lightweight goroutines make it easy to handle many simultaneous operations. A goroutine is essentially a lightweight thread managed

by the Go runtime. For example, handling multiple HTTP requests (user logins, inventory checks) can be done in parallel without complex threading code. This suits the requirement of handling concurrent customers.

- **Built-in Networking and Performance:** Go's standard library includes net/http, allowing us to build RESTful APIs without external frameworks. It also offers efficient performance due to static typing and compilation to native code. In a high-transaction system (e-commerce checkout), these characteristics ensure low latency and reliable service.
- **Error Handling and Reliability:** Go's explicit error handling style forces us to check for issues (e.g., database errors, invalid input) at each step, improving reliability. For a critical path like order processing, explicitly handling errors prevents unexpected failures.
- **Microservice Suitability:** Go binaries are small and self-contained, making them easy to deploy as separate services (possibly in Docker). Its simplicity (no complex runtime) aligns well with a microservice approach where each component is independent.

Justification for Python

- **Asynchronous I/O:** Python's asyncio library (part of the standard library) provides async/await support for concurrent I/O operations. The background services (invoice generator, notification logger, recommendation updater) are largely I/O-bound (reading the event, writing files), so asyncio allows writing them without blocking. For example, an async def listen_events() coroutine can wait on Redis messages and process them as they arrive.
- **Rich Ecosystem:** Python has many libraries for tasks like PDF creation (e.g., ReportLab or WeasyPrint) and data handling. Using Python for the Invoice service means we can quickly produce a nicely formatted PDF using an existing library, focusing effort on the logic rather than reinventing PDF generation.
- **Rapid Development:** Python's concise syntax and dynamic typing speed up development of prototype services. The recommendation logic may need tweaking; doing this in Python (with live data structures like lists or dictionaries) is straightforward. For an academic project, faster development helps ensure features are completed on time.

- **Data Processing:** Even a simple recommendation feature (e.g., "users who bought X often buy Y") can be implemented with Python's list/dict comprehension. If needed, Python supports libraries like pandas or numpy for data processing. At minimum, we can maintain an in-memory mapping of product co-purchases.

2.3 Key Technologies and Frameworks

- **Go:** We will use the standard library's net/http for building the REST server, and database/sql with a PostgreSQL driver (such as github.com/lib/pq) for database access. We may also use gorilla/mux or similar for URL routing. Go structs will model entities (User, Product, Order), and JSON tags will handle request/response serialization.
- **Python:** We'll use asyncio for event loops, and a Redis client like aioredis for Pub/Sub. For PDF generation, a library like ReportLab or WeasyPrint will be used. Python's venv will isolate dependencies.
- **Redis:** We choose Redis Pub/Sub for message passing because it is lightweight and built-in (no need to run a heavy message broker). Redis decouples publishers and subscribers via channels. For example, the Order service publishes to a channel named "order_events", and Python workers subscribe to that channel to receive events.
- **PostgreSQL:** A robust open-source relational database for storing users, products, orders, etc. It supports transactions to ensure, for instance, that inventory is not oversold.
- **Docker:** Docker containers will encapsulate each service and dependency. Docker automates the deployment of applications within lightweight containers, ensuring consistent environments for Go and Python services.

Component	Technology / Library
Go API services	Go standard library
Go database access	pgx or GORM
Authentication	JWT-based authentication
Message broker	RabbitMQ

Component	Technology / Library
Go RabbitMQ client	streadway/amqp or rabbitmq/amqp091-go
Python background workers	Python 3, aio-pika or pika
PDF invoice generation	ReportLab or WeasyPrint
Email service	SMTP using Python's smtplib
Recommendation logic	Pandas or simple SQL-based ranking
Database	PostgreSQL
Caching	Redis
Containerization	Docker and Docker Compose

3. Objectives & Scope

3.1 Objectives

- **Implement RESTful Go Services:** Create Go microservices for user auth, product catalog, inventory, and order processing. Each service should expose clear REST endpoints. Design the services to handle at least 50 concurrent clients performing operations like browsing products or placing orders.
- **Event-Driven Checkout:** Develop the checkout process in the Go Order Service to verify inventory, create the order, update stock, and publish an OrderCreated event to Redis. Use Go concurrency (goroutines) if needed to handle simultaneous order requests safely.
- **Asynchronous Background Tasks:** Use Redis Pub/Sub to decouple tasks. Build Python services that subscribe to the order events: one service generates a PDF invoice for each order, and another records a notification (e.g., to a log table). These should run without blocking the main order process.

- **Recommendation Engine:** Implement a simple recommendation strategy in Python. For each processed order, update recommendation data (for example, track products often bought together). Provide an API endpoint to retrieve recommendations for a user based on their past orders.
- **Testing and Reliability:** Design the system to handle errors gracefully (invalid input, database failure, insufficient stock). Conduct concurrency testing (e.g., multiple checkout requests for limited stock) to ensure correctness. Demonstrate basic performance (such as orders/sec) to show the system can handle modest load.

3.2 In-Scope Features

- User registration and JWT-based login.
- Product catalogue with product details, price, category, and available stock.
- Add-to-cart, update-cart, and remove-from-cart functionality.
- Inventory availability check and stock reservation during checkout.
- Order creation and order-status storage.
- Event-driven communication using RabbitMQ.
- PDF invoice generation using Python.
- Order-confirmation email notification using Python.
- Basic product recommendation based on purchased product categories.
- REST API documentation and Docker-based local deployment.

3.3 Out-of-Scope Features

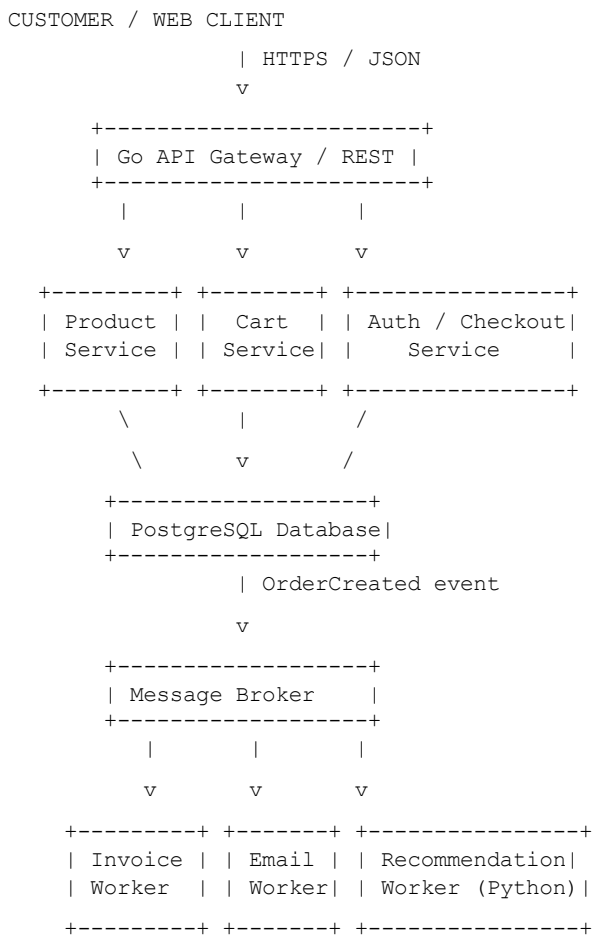
- No full frontend web or mobile application will be developed; APIs will be tested.
- No real payment-gateway integration will be implemented; checkout will use a simulated payment-success flow.
- No advanced machine-learning recommendation system will be built.
- No admin dashboard or graphical reporting interface will be developed.
- No multi-currency, tax-calculation, or international shipping module will be included.
- No advanced fraud detection or large-scale distributed tracing system will be implemented.

4. System Architecture & Design

The proposed architecture is a set of interacting components, designed to keep user-facing performance separate from slower background tasks.

- All client requests go to the **Go API Gateway**, which routes to specific services.
- **Go Microservices** (User, Product, Inventory, Order) handle synchronous operations and talk to **PostgreSQL**.
- After the Order service completes an order, it sends a message to **Redis Pub/Sub**.
- **Python Services** (Invoice, Notification, Recommendation) subscribe to Redis events and process them asynchronously.

4.1 High-Level Architecture Diagram

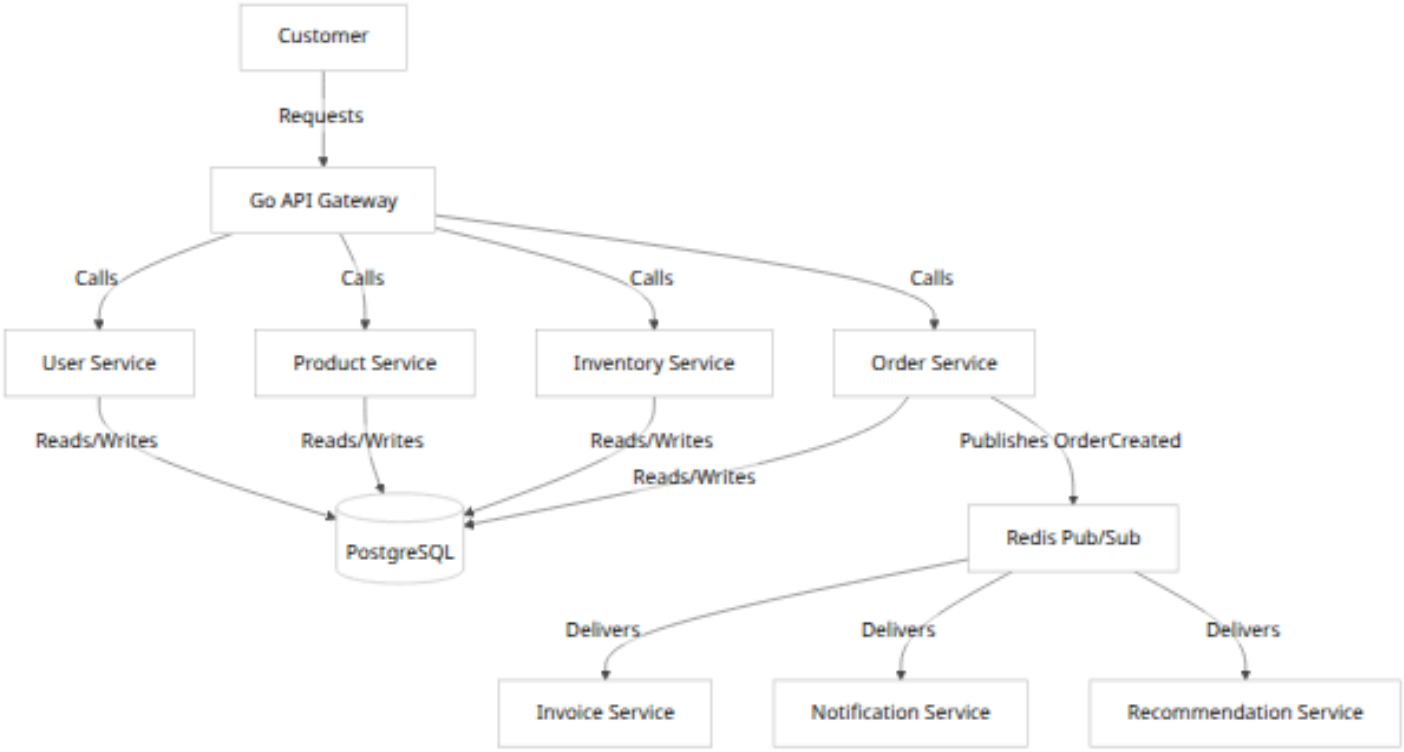


Component Descriptions

- **Go API Gateway:** Entry point for all HTTP requests. It routes requests to the correct service, validates input, and checks authentication tokens. It does not access the database directly.
- **User Service (Go):** Handles user registration, login, and profile retrieval. It stores user details in the `users` table with hashed passwords.
- **Product Service (Go):** Manages the product catalogue. It supports adding, updating, and fetching product details, storing data in the `products` table.
- **Inventory Service (Go):** Checks and updates product stock. During checkout, it verifies availability and reduces stock using database transactions to prevent overselling.
- **Order Service (Go):** Coordinates checkout. It validates the cart, reserves stock, calculates the total, simulates payment, creates order records, and publishes an OrderCreated event. All critical steps are handled in a transaction so failures can be rolled back.
- **PostgreSQL:** Main relational database storing users, products, inventory, orders, order items, and notification/recommendation data. It supports ACID transactions for reliable order processing.
- **Redis Pub/Sub:** In-memory message broker. The Go Order Service publishes order events, while Python services subscribe and process them independently, keeping background work separate from checkout.
- **Invoice Service (Python):** Listens for order events and generates a PDF invoice using ReportLab. The invoice is saved to disk for demonstration.
- **Notification Service (Python):** Consumes order events and records a notification such as “Order #1234 completed for user 45.” In production, this could send an email or SMS.

- **Recommendation Service (Python):** Processes order events to identify frequently bought products together. It updates recommendation records in the database using simple counters and Python data structures.

4.2 Data Flow Diagram



4.3 Input–Process–Output (IPO)

- **Input:** HTTP requests (with JSON payloads) from clients, including user credentials, product IDs/quantities, and commands (e.g., checkout). Configuration inputs (DB connection strings, Redis channel names) come from environment variables or config files. Historical data (past orders) may be used by the recommendation service.
- **Process:** The system validates inputs (check user login, input format), performs business logic (inventory checks, price calculation), executes database transactions (creating users, orders, etc.), and uses pub/sub messaging. Go services use goroutines to handle requests

concurrently. Python services use asyncio to wait for and process events. Error checks at each step ensure invalid operations are rejected (returning HTTP error codes).

- **Output:** REST API responses (JSON objects confirming actions or showing requested data). Database updates (new user, new order, decremented inventory). **After** order placement, the output includes an OrderCreated event to Redis. Then, output artifacts include a PDF invoice file and entries in a notification log or recommendation table. Error messages (JSON) are returned for invalid inputs.

4.4 Language-Specific Features

- **Go:** We will use goroutines for concurrent request handling (e.g., go handleOrder(...)). Channels may be used internally if needed for synchronization. Go structs will model data (User, Product, etc.). Interfaces might abstract the database layer for testability. JSON marshaling is built-in for HTTP. Go's defer and error return values will be used for resource cleanup and error handling.
- **Go Concurrency:** A goroutine is a lightweight thread of execution. We might spawn a new goroutine for each incoming request or long-running DB call. Channels can synchronize any needed coordination (though most communication here is via Redis, not direct channel usage).
- **Python:** Background services will likely use asyncio with async def functions. The Redis client (e.g., aioredis) uses async APIs to subscribe to channels without blocking.
- **Data Classes:** We may define Python classes or dataclasses for invoice or order data, but since we're reading JSON or DB results, plain dicts may suffice.
- **Multiprocessing (optional):** If generating a PDF is CPU-intensive, we could use concurrent.futures.ProcessPoolExecutor for parallelism. However, given the scope, a simple asyncio sequential processing may be enough.
- **Error Handling:** Python services should catch exceptions (e.g., database or file I/O errors) and log them, ensuring one failed invoice generation doesn't crash the service.
- **Database Schema:** Key fields include SERIAL primary keys, VARCHAR for names/emails, TIMESTAMP for creation times. We will use FOREIGN KEY constraints (e.g., order_items referencing orders and products). The schema design will follow normalization best practices.

5. Work Plan & Timeline

A phased plan ensures progress from design to implementation and testing.

Milestone	Timeline	Work	Deliverable
Milestone 1	1 week	Finalize requirements, draw architecture diagrams, design DB schema, write this report.	Phase I Report
Milestone 2	2 weeks	Setup GitHub repo, Docker environment, initialize PostgreSQL, scaffold Go projects, define API contracts.	Initial project skeleton
Milestone 3	3 weeks	Implement User, Product, and Inventory services in Go; create REST endpoints; unit test basic operations.	Core Go services
Milestone 4	1-2 weeks	Implement Order Service with checkout logic; integrate Redis; publish OrderCreated events; test order flow.	Go Order Service + Redis Pub/Sub
Milestone 5	2 weeks	Develop Python background services (Invoice, Notification, Recommendation); integrate with Redis subscriptions; test asynchronous tasks.	Background processing services
Milestone 6	2 weeks	Perform integration testing (Go ↔ Redis ↔ Python), concurrency testing (simultaneous orders), and fix bugs.	Tested system prototype
Milestone 7	1 week	Finalize documentation, diagrams, prepare demonstration, and review for viva.	Phase III final deliverable

Phase II Deliverables: By the Phase II deadline, we expect a working prototype of the backend: Go services for user/product/inventory/order, Redis running, and at least one Python consumer processing orders. We will demonstrate creating users/products and processing an order end-to-end.

Phase III Deliverables: Phase III will complete the system: full background tasks, thorough testing reports, performance measures (concurrent orders, etc.), and final documentation. The viva will involve explaining the architecture and showing the system in action.

6. References

- McGranaghan, M., & Bendersky, E. (2013). *Go by Example: Goroutines*. Retrieved October 22, 2026, from https://gobyexample.com/goroutines
- Python Software Foundation. (2026). *asyncio — Asynchronous I/O*. Python 3.15.1 documentation. Retrieved October 22, 2026, from https://docs.python.org/3/library/asyncio.html
- Redis. (2026). *Pub/Sub*. Redis Documentation. Retrieved October 22, 2026, from https://redis.io/docs/latest/develop/pubsub/
- Docker (software). (2026). *Wikipedia*. Retrieved October 22, 2026, from https://en.wikipedia.org/wiki/Docker_(software)